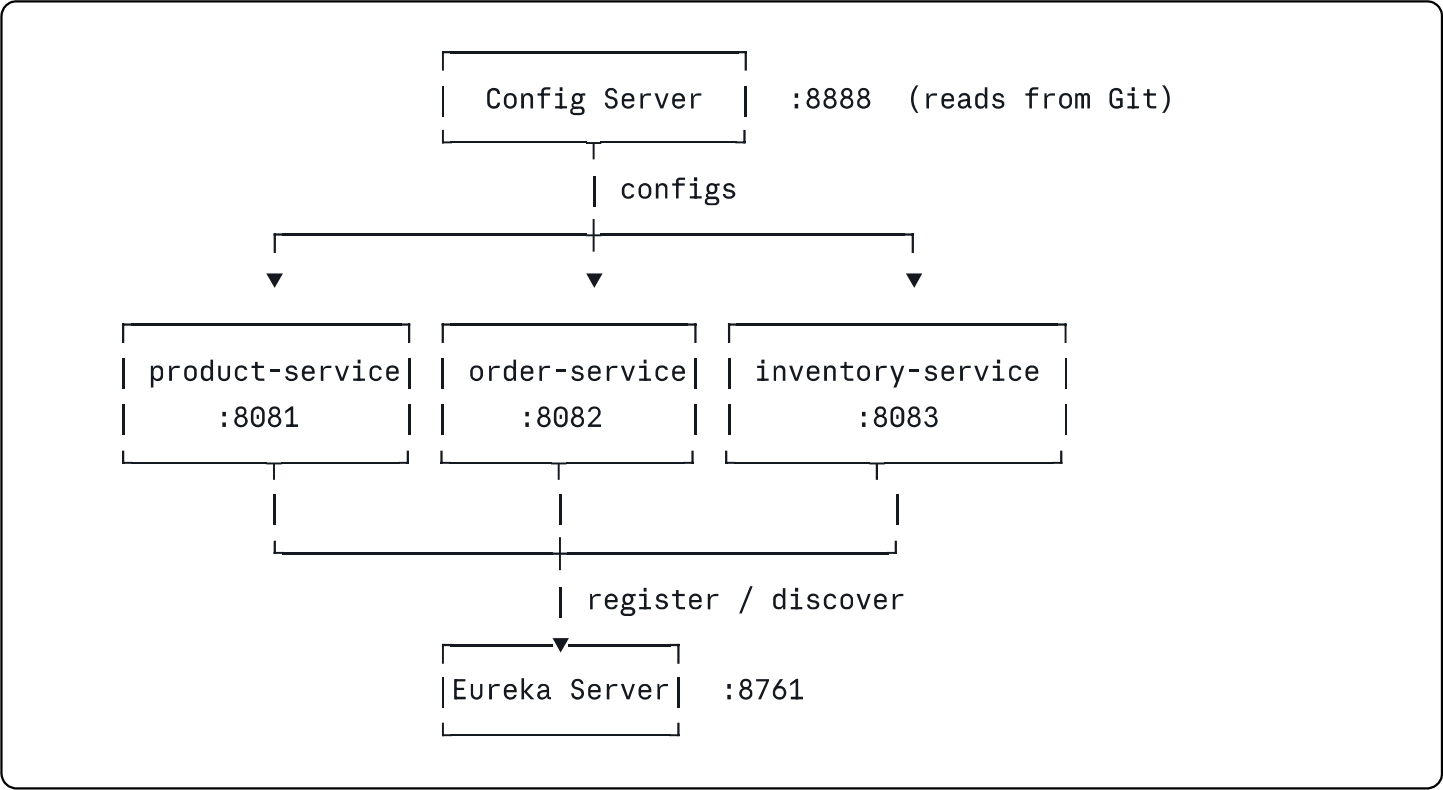


Microservices Setup: Order, Product, Inventory

Architecture Overview



Order Service uses Spring Cloud Load Balancer to call Product & Inventory by service name.

1. Config Server

`pom.xml`

```
xml
```

```

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.5</version>
</parent>

<properties>
  <spring-cloud.version>2023.0.1</spring-cloud.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-config-server</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

ConfigServerApplication.java

```

java

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

```

application.yml

```
yaml

server:
  port: 8888

spring:
  application:
    name: config-server
  cloud:
    config:
      server:
        git:
          uri: https://github.com/your-username/config-repo    # your git repo
          default-label: main
          clone-on-start: true
          # For private repos, add:
          # username: your-github-username
          # password: your-github-token
```

2. Eureka Server

pom.xml

```
xml

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

EurekaServerApplication.java

```
java
```

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

application.yml

```
yaml

server:
  port: 8761

spring:
  application:
    name: eureka-server

eureka:
  client:
    register-with-eureka: false    # don't register itself
    fetch-registry: false
  server:
    wait-time-in-ms-when-sync-empty: 0
```

3. Git Config Repo

Create a GitHub repo (e.g., `config-repo`) with these files:

product-service.yml

```
yaml
```

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:postgresql://product-db:5432/product-db
    username: product_user
    password: product_pwd
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    database-platform: org.hibernate.dialect.PostgreSQLDialect

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
  instance:
    prefer-ip-address: true
```

order-service.yml

```
yaml

server:
  port: 8082

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
  instance:
    prefer-ip-address: true
```

inventory-service.yml

```
yaml
```

```
server:
  port: 8083

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
  instance:
    prefer-ip-address: true
```

4. Product Service

pom.xml

```
xml

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>    <!-- no runtime scope - Jib fix -->
  </dependency>
</dependencies>
```

application.yml (local only — rest comes from config server)

yaml

```
spring:
  application:
    name: product-service          # must match config repo filename
  config:
    import: "configserver:http://localhost:8888"
```

ProductController.java

java

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Value("${server.port}")
    private String port;

    @GetMapping
    public Map<String, Object> getAllProducts() {
        return Map.of(
            "service", "product-service",
            "servedByPort", port,
            "products", List.of(
                Map.of("id", 1, "name", "Laptop", "price", 999.99),
                Map.of("id", 2, "name", "Phone", "price", 499.99)
            )
        );
    }

    @GetMapping("/{id}")
    public Map<String, Object> getProduct(@PathVariable int id) {
        return Map.of(
            "id", id,
            "name", "Laptop",
            "price", 999.99,
            "servedByPort", port
        );
    }
}
```

5. Inventory Service

pom.xml

xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

application.yml

yaml

```
spring:
  application:
    name: inventory-service
  config:
    import: "configserver:http://localhost:8888"
```

InventoryController.java

java


```

@RestController
@RequestMapping("/api/inventory")
public class InventoryController {

    @Value("${server.port}")
    private String port;

    @GetMapping
    public Map<String, Object> getAllInventory() {
        return Map.of(
            "service", "inventory-service",
            "servedByPort", port,
            "items", List.of(
                Map.of("productId", 1, "quantity", 100, "available", true),
                Map.of("productId", 2, "quantity", 50, "available", true)
            )
        );
    }

    @GetMapping("/{productId}")
    public Map<String, Object> getInventoryByProduct(@PathVariable int productId) {
        return Map.of(
            "productId", productId,
            "quantity", 100,
            "available", true,
            "servedByPort", port
        );
    }
}

```

6. Order Service

This is the key service — it uses `@LoadBalanced` `WebClient` to call the other two services by name.

`pom.xml`

xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- WebClient lives in webflux -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
  </dependency>
  <!-- Load balancer is transitively included via eureka-client,
       but declare it explicitly for clarity -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-loadbalancer</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

application.yml

```
yaml

spring:
  application:
    name: order-service
  config:
    import: "configserver:http://localhost:8888"
```

OrderServiceApplication.java

```
java
```

@SpringBootApplication

```
public class OrderServiceApplication {

    // @LoadBalanced tells Spring Cloud to intercept calls and
    // resolve http://product-service/... via Eureka + Load Balancer
    @Bean
    @LoadBalanced
    public WebClient.Builder loadBalancedWebClientBuilder() {
        return WebClient.builder();
    }

    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

OrderController.java

java

```

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final WebClient.Builder webClientBuilder;

    public OrderController(WebClient.Builder webClientBuilder) {
        this.webClientBuilder = webClientBuilder;
    }

    @GetMapping
    public Map<String, Object> placeOrder() {

        // Spring Cloud Load Balancer resolves "product-service"
        // to one of its registered instances in Eureka
        Map<?, ?> product = webClientBuilder.build()
            .get()
            .uri("http://product-service/api/products/1")
            .retrieve()
            .bodyToMono(Map.class)
            .block();

        Map<?, ?> inventory = webClientBuilder.build()
            .get()
            .uri("http://inventory-service/api/inventory/1")
            .retrieve()
            .bodyToMono(Map.class)
            .block();

        return Map.of(
            "orderId",    UUID.randomUUID().toString(),
            "status",     "CREATED",
            "product",    product,
            "inventory",  inventory
        );
    }
}

```

7. Docker Compose (full stack)

yaml

```
version: '3.8'
```

```
services:
```

```
  config-server:
```

```
    image: hkc7792/config-server:v1
```

```
    container_name: config-server
```

```
    ports:
```

```
      - "8888:8888"
```

```
    networks:
```

```
      - backend-network
```

```
    healthcheck:
```

```
      test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"]
```

```
      interval: 10s
```

```
      timeout: 5s
```

```
      retries: 5
```

```
  eureka-server:
```

```
    image: hkc7792/eureka-server:v1
```

```
    container_name: eureka-server
```

```
    ports:
```

```
      - "8761:8761"
```

```
    depends_on:
```

```
      config-server:
```

```
        condition: service_healthy
```

```
    networks:
```

```
      - backend-network
```

```
  product-db:
```

```
    image: postgres:16-alpine
```

```
    container_name: product-db
```

```
    environment:
```

```
      POSTGRES_DB: product-db
```

```
      POSTGRES_USER: product_user
```

```
      POSTGRES_PASSWORD: product_pwd
```

```
    volumes:
```

```
      - product_postgres_data:/var/lib/postgresql/data
```

```
    networks:
```

```
      - backend-network
```

```
  product-service:
```

```
    image: hkc7792/product-service:v1
```

```
    container_name: product-service
```

```
    ports:
```

```
      - "8081:8081"
```

```
    environment:
```

```
    SPRING_CONFIG_IMPORT: "configserver:http://config-server:8888"
depends_on:
  - config-server
  - eureka-server
  - product-db
networks:
  - backend-network
```

```
inventory-service:
  image: hkc7792/inventory-service:v1
  container_name: inventory-service
  ports:
    - "8083:8083"
  environment:
    SPRING_CONFIG_IMPORT: "configserver:http://config-server:8888"
  depends_on:
    - config-server
    - eureka-server
  networks:
    - backend-network
```

```
order-service:
  image: hkc7792/order-service:v1
  container_name: order-service
  ports:
    - "8082:8082"
  environment:
    SPRING_CONFIG_IMPORT: "configserver:http://config-server:8888"
  depends_on:
    - config-server
    - eureka-server
    - product-service
    - inventory-service
  networks:
    - backend-network
```

```
networks:
  backend-network:
    driver: bridge
```

```
volumes:
  product_postgres_data:
```

Startup Order

Always start in this sequence — services will fail if config/eureka aren't ready first:

1. config-server → wait for :8888/actuator/health = UP
 2. eureka-server → wait for :8761 dashboard
 3. product-service, inventory-service (parallel OK)
 4. order-service
-

Verify It Works

```
bash

# All registered in Eureka?
open http://localhost:8761

# Config server serving configs?
curl http://localhost:8888/product-service/default
curl http://localhost:8888/order-service/default

# Individual services
curl http://localhost:8081/api/products
curl http://localhost:8083/api/inventory

# Order service calls the other two via load balancer
curl http://localhost:8082/api/orders
```

The response from `/api/orders` will contain live data fetched from product-service and inventory-service, resolved through Eureka by service name.